The product is an education-only video platform, not a general entertainment feed. Users explicitly choose subjects and topics. The home feed prioritises those interests and avoids unrelated recommendations. Users can explore other subjects deliberately, but exploration should not silently take over the main feed.

The product's distinctive value is the combination of topic-aware discovery, evidence-informed ranking, editable learning playlists, and an in-context workspace. A learner can watch an embedded video, write code, take timestamped personal notes, generate AI notes from authorised material, and save everything into an organised library.

**Product principle: AI organises and assists; users retain control. A language model should not be the live ranking engine for every feed request.**

## 1.1 Product goals

- Reduce time lost to unrelated video recommendations.
- Make it easy to find the strongest explanation for a specific concept, not just a popular video.
- Show why a video or playlist is recommended and how confident the platform is.
- Support active learning through code practice, notes, timestamps and saved study material.
- Let users create, customise, share and rate learning playlists.
- Build an architecture that can start as a solo-developer MVP and scale incrementally.

## 1.2 Non-goals for the first release

- Do not classify users as beginner, intermediate or advanced, or force school/college education-stage filters.
- Do not rehost YouTube videos or assume unrestricted access to YouTube captions.
- Do not build a full LMS, live classroom, social network, or multi-language cloud IDE in the first MVP.
- Do not use an LLM to generate a fresh ranking for every page view.
- Do not let watch time alone redefine a user's interests.

## 1.3 Product success measures

| Metric | Definition | Why it matters |
|---|---|---|
| Relevant-video rate | Share of impressions users mark useful or engage with meaningfully | Measures feed relevance |
| Topic satisfaction | Rating and feedback for a topic page | Measures whether topic discovery works |
| Playlist completion | Users reaching the end of a saved/started learning path | Measures sequence usefulness |
| Time-to-understanding proxy | Time from topic search to save, practice completion or positive feedback | Measures reduced discovery friction |
| Workspace adoption | Share of video sessions using notes, code or AI notes | Measures product differentiation |
| AI correction rate | Share of AI classifications/notes edited or reported | Measures AI reliability |

# 2. Product principles and user journeys

## 2.1 Core experience

A user selects broad subjects, fields and specific topics. They may follow a broad subject such as Computer Science, a field such as Web Development, or a narrow topic such as React Hooks. Selection is optional at each depth: do not force every user through a long taxonomy wizard.

- Home: For You feed based primarily on explicitly selected topics.

- Topics: browse a hierarchy and open topic-specific video rankings.
- Playlists: AI-curated sequences and user-created/customised playlists.
- Explore: deliberate discovery outside followed topics; keep separate from the main feed.
- Library: saved playlists, notes, AI notes, code snippets and folders.
- Profile/settings: manage followed topics, muted topics, privacy and recommendation controls.

### 2.2 Example learner journey

| Step | User action | Platform response |
|---|---|---|
| 1 | Selects Computer Science, DSA, React and System Design | Stores explicit interests and their granularity |
| 2 | Opens DSA → Trees → Binary Search Tree | Shows topic overview, top explanations and playlists |
| 3 | Chooses a video | Opens embedded player and contextual workspace |
| 4 | Writes code and timestamped notes | Autosaves drafts and links them to video/topic |
| 5 | Requests AI notes | Creates structured notes from permitted transcript/material |
| 6 | Saves notes to DSA / Trees folder | Adds item to personal library |
| 7 | Rates video and playlist | Updates confidence-adjusted community signals |

### 2.3 Avoiding recommendation distraction

The main feed should be governed by explicit interest choices. Add an Explore surface for adjacent subjects. A user can opt into related-topic recommendations or temporarily explore without permanently changing their followed topics. Provide 'Not interested', mute topic, unfollow topic and reset recommendations controls.

Watch duration is a weak and ambiguous signal: a video may be left playing in the background. Give stronger weight to explicit actions such as follow topic, save, add to playlist, positive rating, 'more like this' and 'not interested'. Use behavioural signals as supporting evidence, not as the only definition of interest.

## 3. Subject taxonomy and content model

The platform classifies a video by what it actually teaches, not merely by the category selected by the original uploader. A video may belong to multiple topics. Store one primary topic plus secondary topic labels and per-topic coverage.

| Broad subject | Example fields / topics |
|---|---|
| Computer Science & IT | Programming, Web Development, Mobile, DSA, System Design, Databases, AI, ML, Cybersecurity, Cloud, DevOps, Networks |
| Mathematics | Algebra, Geometry, Calculus, Statistics, Probability, Discrete Mathematics, Applied Mathematics |
| Science | Physics, Chemistry, Biology, Astronomy, Environmental Science, Biotechnology |
| Business & Finance | Entrepreneurship, Marketing, Accounting, Economics, Management, Financial literacy, investing education |
| Humanities & Social Sciences | History, Geography, Psychology, Philosophy, Political Science, Sociology |
| Languages & Communication | English, Hindi, other languages, grammar, writing, public speaking, communication |

| Broad subject | Example fields / topics |
|---|---|
| Engineering & Technology | Mechanical, Civil, Electrical, Electronics, Aerospace, Robotics |
| Medical & Health Education | Anatomy, Physiology, Pharmacology, Public Health, Nursing |
| Law & Government | Constitutional, Criminal and Civil Law, Governance, Public Administration |
| Arts & Creative Learning | Graphic Design, Photography, Animation, Music Theory, Video Editing, Creative Writing |

### 3.1 Topic hierarchy

Use stable topic IDs, not topic names, as database references. Names can change; IDs should not. Store aliases such as 'JS' and 'JavaScript' for search and mapping. Support topic merging and redirects when the taxonomy changes.

### 3.2 Cross-topic videos

A single video can explain React Hooks generally, then spend a chapter on useEffect and another on useMemo. Store topic coverage by segment where possible. The video can appear in multiple topic pages, but its score should be computed for the topic currently being viewed.

### 3.3 Content classification confidence

AI classification should return confidence and evidence. Low-confidence or conflicting classifications should enter a review queue or be labelled provisional. Never treat a model's confident wording as proof of correctness.

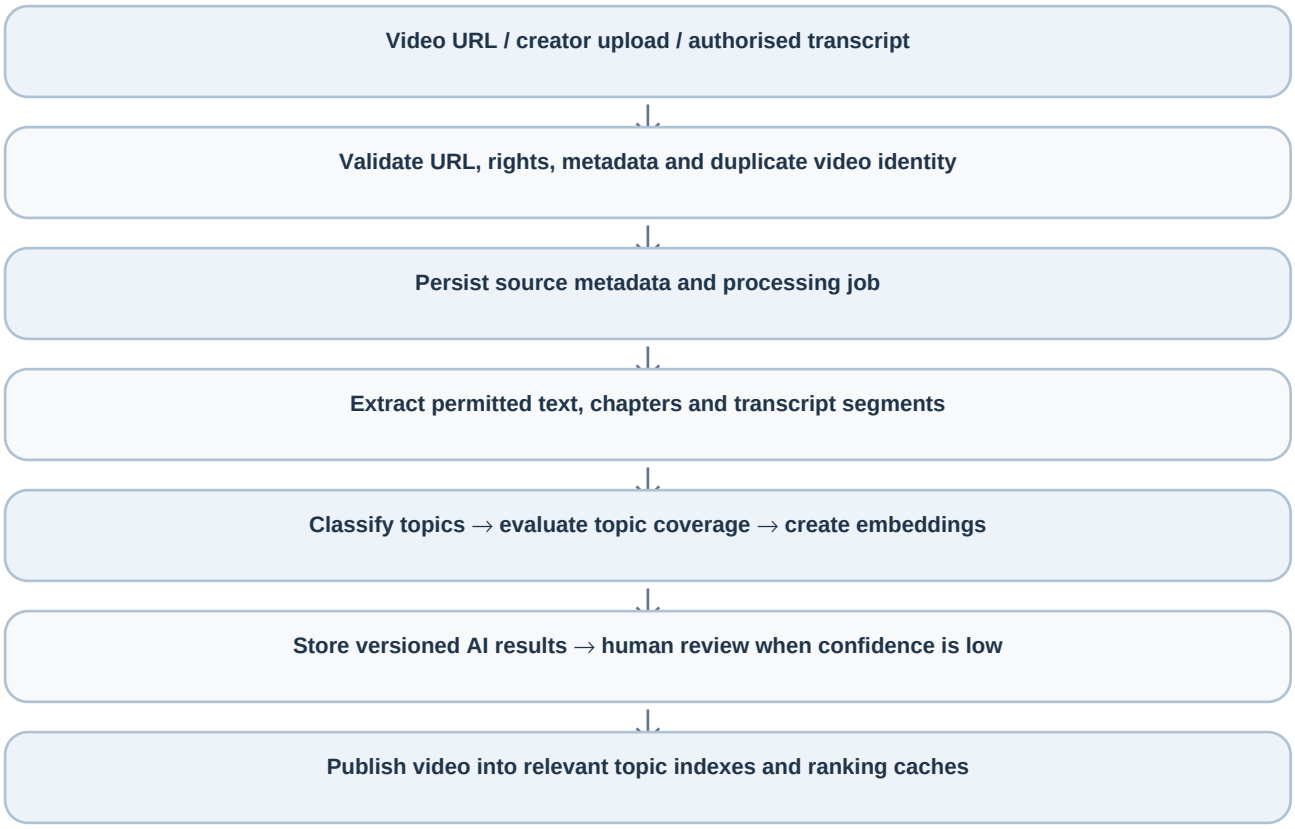
## 4. Video ingestion, metadata and rights

### 4.1 Initial content sources

- YouTube public metadata retrieved through permitted YouTube Data API operations.
- YouTube video playback embedded using the IFrame Player API.
- Creator-submitted or creator-authorised transcripts and learning materials.
- User-provided files where the user has rights to upload and process them.
- Later: direct creator uploads, licensed course feeds or partner content.

## 4.2 Ingestion pipeline

### ASYNCHRONOUS VIDEO PROCESSING




## 4.3 Video record fields

| Field group | Suggested fields |
|---|---|
| Identity | _id, provider, providerVideoId, canonicalUrl, creatorId |
| Metadata | title, description, thumbnailUrl, durationSeconds, publishedAt, language |
| Classification | primaryTopicId, topicIds, topicCoverage[], classificationConfidence |
| AI assessment | evaluationVersion, clarity, accuracyEvidence, exampleQuality, completeness, limitations |
| Rights | sourceTerms, transcriptSource, processingPermission, retentionPolicy |
| Operational | ingestionStatus, lastProcessedAt, processingError, createdAt, updatedAt |

## 4.4 YouTube-specific constraints

Treat YouTube as a video source and playback provider, not as a source from which the platform can freely download and store everything. The IFrame Player API supports embedded playback and player controls. The YouTube Data API has quota limits and developer policies. Caption access is permission-sensitive; do not build the product around scraping or assuming public caption downloads.

Cache permitted metadata in accordance with the applicable YouTube API policies, retain attribution and links, and review current terms before launch. If you cannot lawfully obtain a transcript, still allow video discovery and playback, but disable transcript-dependent AI functions or use authorised creator-provided material.

Reference: YouTube Data API overview and developer policies are listed in the references section.

# 5. Video ranking and recommendation engine

## 5.1 Separate the ranking dimensions

| Dimension | Question answered | Stored at |
|---|---|---|
| Video rating | How useful did viewers find this video overall? | Video or rating aggregate |
| Topic-specific rating | How well did it explain this topic? | Video + topic aggregate |
| AI topic evaluation | What does the content appear to cover and how well? | Video + topic + evaluation version |
| Playlist rating | How useful is this ordered sequence? | Playlist aggregate |
| Playlist quality | Does the sequence cover concepts coherently? | Playlist evaluation/version |

## 5.2 User rating model

Use a five-star or simple 1–5 rating, optionally accompanied by structured feedback: clarity, practical examples, relevance to the selected topic, accuracy concerns and whether the video answered the question. Keep rating friction low and permit skip. Prevent duplicate ratings from the same account for the same rating target; allow later edits with audit history.

## 5.3 Confidence-adjusted rating

A raw average over-promotes videos with very few ratings. Use a Bayesian average:

```
AdjustedRating = (v / (v + m)) * A + (m / (v + m)) * C

A = observed average rating for this video/topic
v = number of valid ratings
m = confidence threshold (tunable)
C = prior average for comparable videos in the same topic
```

For a new video, the score stays near the topic prior until enough people rate it. Choose m through offline evaluation rather than guessing. Consider Wilson lower bound for binary helpful/not-helpful votes; Bayesian average is intuitive for star ratings.

## 5.4 Topic ranking score

```
TopicScore =
  w_user * normalized_adjusted_user_rating
+ w_ai * normalized_ai_topic_quality
+ w_coverage * topic_coverage
+ w_fresh * freshness_adjustment
+ w_trust * source_or_creator_trust

Apply penalties for:
  - reported inaccuracies awaiting review
  - duplicate or near-duplicate content
  - policy violations
  - misleading metadata
```

Starting point for experimentation: user rating 50–65%, AI evaluation 15–25%, topic coverage 10–20%, and freshness/trust making up the remainder. These are initial hypotheses, not universal constants. Run offline tests and small controlled experiments before choosing weights.

### 5.5 Personalisation without losing user control

- Main feed eligibility: explicit followed topics first.
- Within a topic: sort using topic-specific score and user's chosen sort mode.
- Related topics: only show when the user opts in or in a clearly separate Explore area.
- Diversity: avoid showing ten near-identical videos from one creator consecutively.
- New content: reserve a small, clearly labelled slot for new/unrated videos so incumbents do not permanently dominate.
- Feedback: 'Not relevant', 'Already know this', 'More like this', mute and unfollow.

### 5.6 Ranking explanations

Store reason codes such as 'high learner rating', 'covers useEffect in depth', 'strong practical examples', 'new and not yet rated'. Expose a short 'Why this is recommended' explanation. Do not expose private individual rating data.

### 5.7 Anti-manipulation

- One active rating per user per target; retain edit history.
- Rate limits, suspicious velocity detection and bot checks.
- Weight rating confidence, not raw rating volume.
- Detect rating brigades and unusual account clusters.
- Do not allow creators to buy ranking position.
- Separate sponsored placements from organic ranking and label them clearly if ever introduced.

## 6. AI content intelligence and evaluation

### 6.1 AI jobs should be separate

| AI service | Purpose | Trigger |
|---|---|---|
| Topic classifier | Map content to subject/field/topic IDs | New or updated video |
| Segment extractor | Identify timestamps and concept boundaries | Transcript/chapter ingestion |
| Topic coverage evaluator | Estimate which concepts are actually explained | New topic mapping or re-evaluation |
| Teaching quality evaluator | Assess clarity, examples, completeness and evidence | Background job |
| Playlist sequence planner | Order videos to cover concepts with minimal gaps | Playlist creation/update |
| Notes generator | Create structured notes tied to timestamps | User request or creator workflow |
| Learning assistant | Answer questions grounded in authorised material | User interaction |
| Content moderation/checker | Flag unsafe, spammy or misleading material for review | Ingestion and reports |

### 6.2 Evidence-first evaluation

The model must distinguish evidence from inference. For each assessed concept, request transcript segment references or timestamps. If evidence is missing, return 'unknown' rather than inventing a score. The model should not certify factual accuracy merely because a transcript sounds plausible. Accuracy checks need reliable references, test execution where appropriate, or expert review.

## 6.3 Example evaluator prompt

```
SYSTEM:
You are an educational-content assessment service.
Evaluate only the supplied authorised transcript, chapters and materials.
Do not infer that a concept is explained merely because it appears in the title.
Do not invent timestamps, examples, citations or facts.
If evidence is insufficient, mark the field unknown and lower confidence.
Return only the required JSON schema.

USER INPUT:
Target topic: React Hooks
Concept checklist:
- useState
- useEffect
- dependency array
- cleanup
- useMemo
- useCallback
- custom hooks

Material:
[transcript segments with start/end timestamps]
[chapter metadata]
[creator-provided code examples]

TASK:
For each concept, report:
- coverage: none / mentioned / explained / demonstrated
- evidence segment IDs
- explanation clarity (0-100 or unknown)
- practical example quality (0-100 or unknown)
- potential inaccuracies (quote segment IDs; do not fabricate)
- confidence
- limitations and missing concepts
```

## 6.4 Structured output

Use JSON Schema-constrained outputs where supported, validate every result on your server, and version prompts and schemas. OpenAI Structured Outputs can enforce schema adherence, but schema-valid output can still contain incorrect content. Keep eval datasets and human review for quality.

```
{
  "videoId": "vid_123",
  "topicId": "react_hooks",
  "evaluationVersion": "topic-eval-v1",
  "coverage": [
    {
      "conceptId": "use_effect",
      "level": "demonstrated",
      "evidenceSegmentIds": ["seg_12", "seg_13"],
      "confidence": 0.91
    }
  ],
  "scores": {
    "clarity": 88,
    "examples": 92,
    "completeness": 76
  },
  "limitations": ["useCallback is not covered"],
  "needsHumanReview": false
}
```

## 6.5 Embeddings and semantic matching

Create embeddings for transcript chunks, descriptions and topic definitions. Use them to retrieve candidate topics and related content, then ask a classifier to confirm. Embeddings are useful for semantic similarity, not a substitute for topic taxonomy or quality assessment. Store embedding model/version and re-embed when the model or content changes.

## 6.6 Cost controls

- Process new videos asynchronously; do not run all AI jobs on page load.
- Deduplicate by provider video ID, transcript hash and evaluation version.

- Cache outputs and re-run only when source material or prompt version changes.
- Use smaller/cheaper models for extraction and classification where evaluation proves adequate.
- Use batch processing for offline classification and embedding jobs that do not need immediate results.
- Set per-user quotas for expensive AI actions and show usage limits transparently.
- Track cost per processed video, generated note and active learner.

# 7. Playlist system

## 7.1 Two playlist types

| Type | Owner | Behaviour |
|---|---|---|
| AI-ranked topic playlist | Platform-generated | Selects videos for a topic and orders them using coverage, quality and learner signals |
| User-created playlist | User | User adds, removes and reorders videos; can be private, unlisted or public |
| Customised AI playlist | User-owned copy or fork | Starts from an AI recommendation and lets the user adjust order and selection |

## 7.2 Playlist generation algorithm

- Input: target topic, subtopic map, user-selected preferences, candidate videos and evidence-backed coverage.
- Filter: remove unavailable, duplicate, restricted or low-confidence items.
- Coverage: identify which concepts each video explains and at what depth.
- Sequence: order foundational concepts before dependent concepts when the topic structure supports dependencies.
- Redundancy: penalise repeated explanations unless repetition provides a useful alternative explanation.
- Gaps: detect concepts not covered and label them honestly.
- Output: ordered video IDs, per-item purpose, coverage map, score and explanation.
- User controls: reorder, remove, add, save a copy, publish or keep private.

## 7.3 Playlist quality is not average video quality

A playlist is a sequence. Its usefulness depends on concept coverage, order, redundancy, gaps, consistency and total length. A playlist with five excellent but repetitive videos may be worse than a sequence with four complementary videos.

### 7.4 Example playlist planner prompt

```
SYSTEM:
Build a topic-focused learning sequence from the supplied candidate videos.
Use only supplied metadata and evidence-backed topic coverage.
Do not invent missing content. Prefer coherent progression and low redundancy.
Return the ordered list, rationale, coverage gaps and confidence.

INPUT:
Target topic: Binary Search Trees
Required concepts:
- BST property
- search
- insertion
- deletion
- traversal
- complexity
Candidate videos:
[{videoId, duration, topicCoverage, evidence, learnerScore, confidence}]

OUTPUT:
{
  "orderedVideoIds": [],
  "itemReasons": [],
  "coveredConcepts": [],
  "missingConcepts": [],
  "estimatedTotalMinutes": 0,
  "confidence": 0.0
}
```

### 7.5 Playlist ratings

Rate playlist usefulness separately from video quality. Ask whether the order made sense, whether the playlist covered the topic, and whether it contained unnecessary repetition. Aggregate playlist ratings using the same confidence-aware approach.

## 8. Video learning workspace

### 8.1 Workspace layout

Keep the player prominent and the tools in a tabbed side panel or responsive bottom panel. On desktop, use a two-column layout: video and contextual tools. On mobile, use a stacked player with a drawer or tabs. Do not force a full IDE on small screens.

| Panel | Core functionality | MVP scope |
|---|---|---|
| Video | Embedded playback, seek, playback state, timestamps | Required |
| Code practice | Editor, language selector, run, reset, output | JavaScript first; sandboxed execution |
| Personal notes | Rich/plain text, autosave, timestamps, folders | Required |
| AI notes | Generate, preview, edit, save to folder | Required after transcript path |
| Transcript | Search, click timestamp, select segment for notes | Only when authorised transcript exists |
| Ask AI | Answer using current video's permitted context | Later, with citations/evidence |

### 8.2 Video player integration

Use the provider's embedded player. Track current playback time through supported player events. When the user adds a note, store the current timestamp. Clicking a timestamp should seek the player if the embed API supports the action. Respect provider branding, playback requirements and policy.

## 8.3 Code editor

Monaco Editor offers a VS Code-like browser editor, but its official documentation notes it is not supported on mobile browsers. Consider CodeMirror or a simplified editor for mobile. The editor itself does not execute code; execution belongs in a separate sandbox service.

| Component | Responsibility |
|---|---|
| Editor UI | Syntax highlighting, editing, local draft state |
| Workspace API | Save and retrieve user code files/snippets |
| Execution API | Accept code, language and bounded input; return job ID |
| Sandbox worker | Compile/run with CPU, memory, time and filesystem restrictions |
| Result store | Persist status/output briefly; enforce retention policy |

## 8.4 Code execution security

**Never execute arbitrary user code inside your Next.js application process, a normal API container, or a shared production database network.**

- Use a dedicated sandbox such as Judge0 or an independently audited isolation service.
- Disable outbound network by default; restrict filesystem access and process spawning.
- Apply CPU, memory, wall-clock, output-size and concurrent-job limits.
- Use ephemeral containers/VMs and destroy execution environments after completion.
- Keep sandbox credentials server-side; never expose them to the browser.
- Rate-limit runs per user and protect against abuse and denial of service.
- Treat execution output as untrusted text; escape it in the UI.

## 8.5 Personal notes and timestamp links

A note can be attached to a video, a topic, a playlist or no source. Store timestampSeconds when linked to a video. Support draft autosave, explicit saved state, version history where valuable, search, tags, folder movement and export.

## 8.6 AI-generated notes

Generate structured notes from authorised transcript chunks and supplied context. Include title, topic, sections, definitions, code snippets, examples, timestamp references, uncertainty notes and source segment IDs. Make notes editable. Keep AI notes separate from personal notes by default; offer an explicit merge/copy action.

## 8.7 Folders and library

- Default folders: My Notes, AI Notes, Code Snippets, Saved Playlists.
- Users can create, rename, move and delete folders.
- Use a parentFolderId for nested folders only if users need hierarchy; start with one level to reduce complexity.
- Soft-delete with a recovery window if product policy supports it.
- Search across note title, content, tags and topic labels.
- A saved AI note is a user-owned snapshot; regenerating the source note should not silently overwrite the saved copy.

# 9. Frontend information architecture

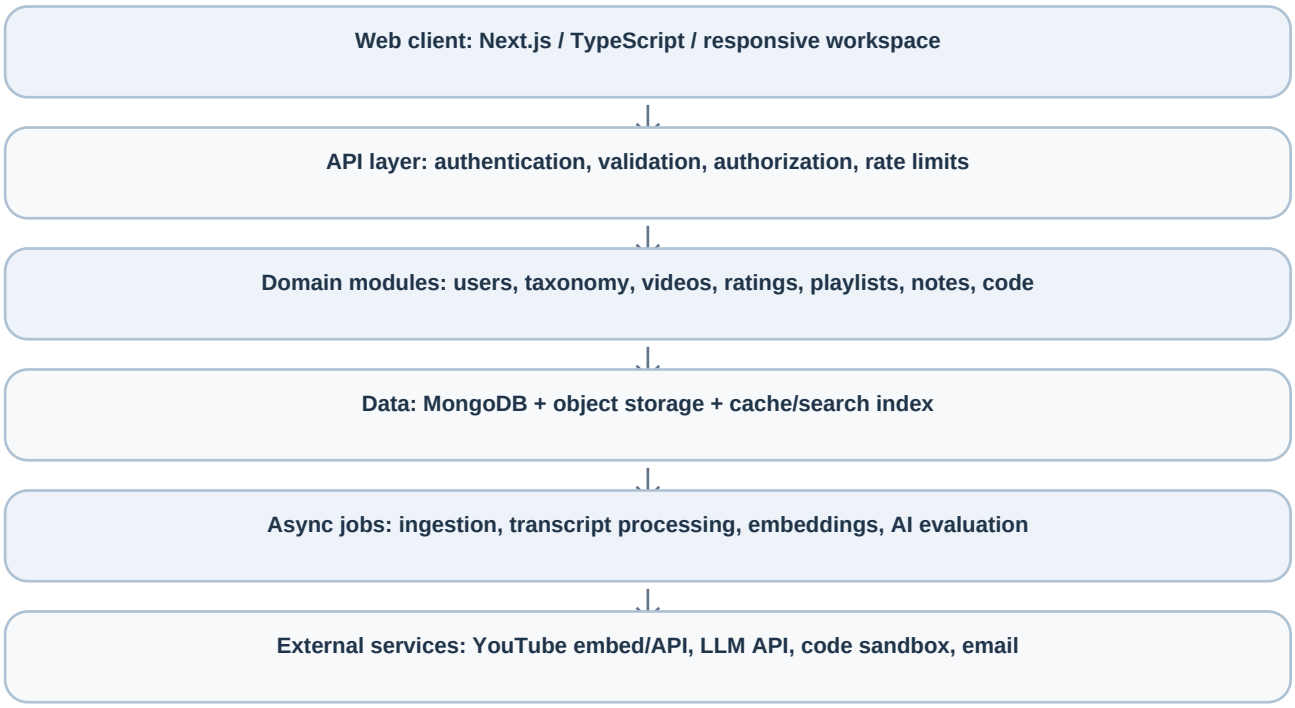
| Page / route | Purpose | Important UI elements |
|---|---|---|
| / | Focused home feed | Followed topics, continue learning, top topic videos |
| /topics | Taxonomy browser | Search subjects, follow/unfollow, topic hierarchy |
| /topics/[slug] | Topic detail | Best videos, related subtopics, AI and community signals |
| /watch/[videoId] | Learning workspace | Player, code, notes, AI notes, transcript |
| /playlists | Playlist library | AI playlists, custom playlists, saved playlists |
| /playlists/[id] | Playlist detail | Ordered videos, coverage, reorder, rate, share |
| /library | Personal library | Folders, notes, code, saved items |
| /search | Search | Video/topic/playlist results with filters |
| /settings/interests | Interest controls | Followed topics, muted topics, feed preferences |
| /admin | Content operations | Review queue, taxonomy, AI job status, reports |

## 9.1 Accessible and responsive UX

- Keyboard-operable player controls, tabs, editor and dialogs.
- Visible focus states, sufficient contrast and semantic headings.
- Captions or transcripts only where authorised and available; communicate when unavailable.
- Respect reduced-motion preferences and avoid autoplay by default.
- Keep the primary learning task usable on narrow screens.
- Provide loading, empty, error, unavailable-video and AI-processing states.

# 10. System design and service architecture

### LOGICAL SYSTEM ARCHITECTURE




## 10.1 Start modular, not microservices

For a solo developer, begin with a modular monolith: one deployable Next.js/Node application with clear domain modules, plus separate background worker and isolated code-execution service. Split services only when scaling, deployment independence or security boundaries justify it.

## 10.2 Suggested technology stack

| Layer | Recommended starting choice | Reason / caveat |
|---|---|---|
| Frontend | Next.js + TypeScript | Routing, server rendering where useful, one primary web app |
| UI | Tailwind CSS + shadcn/ui | Consistent components and fast iteration |
| Client state | Zustand for local/global UI state | Keep server data in query/cache layer, not duplicated in Zustand |
| Server data | TanStack Query or framework-native fetch/cache | Loading, invalidation and request lifecycle |
| Backend | Next.js Route Handlers initially; Node service if needed | Avoid premature service sprawl |
| Database | MongoDB Atlas + Mongoose or native driver | Flexible content and user-owned documents |
| Search | MongoDB Search initially; dedicated search later if needed | Topic, title and note search |
| Cache/queue | Redis-compatible cache/queue when required | Feed cache, rate limits, background jobs |
| AI | OpenAI API with structured outputs and embeddings | Classification, evaluation, notes, assistant |
| Editor | Monaco desktop; responsive alternative on mobile | Monaco mobile browser limitation |
| Execution | Judge0 or equivalent isolated runner | Do not run arbitrary code in app server |
| Storage | Object storage for user uploads and exports | Keep large files out of MongoDB documents |
| Monitoring | Error tracking, logs, metrics and tracing | Measure before optimising |

## 10.3 Domain modules

| Module | Owns |
|---|---|
| Identity | Accounts, sessions, roles, preferences |
| Taxonomy | Subjects, fields, topics, aliases, relationships |
| Catalog | Video provider metadata, ingestion state, availability |
| Content intelligence | Transcript chunks, embeddings, AI evaluations |
| Discovery | Feed candidates, topic ranking, search |
| Ratings | User feedback, aggregates, abuse checks |
| Playlists | AI sequences, custom playlists, playlist items |
| Workspace | Code drafts, editor preferences, execution jobs |
| Notes/library | Personal notes, AI notes, folders, saved items |
| Admin/moderation | Review queues, reports, audit logs |

### 10.4 Synchronous versus asynchronous work

| Synchronous request | Background job |
|---|---|
| Load topic page | Fetch/process new transcript |
| Save note | Generate AI notes |
| Submit rating | Evaluate video quality |
| Reorder playlist | Create embeddings |
| Request code execution job | Run sandbox and collect result |
| Search existing indexed content | Rebuild ranking aggregates |

Return a job ID for long-running work and expose job status. Use idempotency keys for retried jobs. A worker should be able to retry safely without creating duplicate notes, evaluations or playlist items.

# 11. MongoDB data model

The following is a logical model, not a mandate to create every collection on day one. Embed small bounded subdocuments that are always read together; reference large, frequently changing or many-to-many records. Keep large transcripts and files outside hot user documents.

| Collection | Important fields | Notes |
|---|---|---|
| users | _id, email, displayName, role, preferences, createdAt | Never expose private fields in public profile APIs |
| userInterests | userId, topicId, state, source, createdAt | Unique compound index userId + topicId |
| topics | _id, parentId, slug, name, aliases, status | Stable IDs and slug uniqueness |
| videos | _id, provider, providerVideoId, metadata, status | Unique provider + providerVideoId |
| videoTopics | videoId, topicId, coverage, confidence, evidenceRefs | One record per video-topic mapping |
| transcriptSegments | videoId, segmentId, start, end, text, source | Only permitted transcript content |
| aiEvaluations | videoId, topicId, version, scores, evidence, status | Immutable/versioned assessment snapshots |
| videoRatings | userId, videoId, topicId?, score, feedback | Unique user + target + topic scope |
| ratingAggregates | targetId, scope, count, sum, distribution, updatedAt | Update asynchronously or transactionally as needed |
| playlists | ownerId, type, topicId, title, visibility, ratingAggregate | type: AI, USER, CUSTOMISED |
| playlistItems | playlistId, videoId, position, reason, coverage | Unique playlistId + position |
| notes | ownerId, folderId, source, type, title, content, timestamps | Personal and AI notes are distinct types |
| folders | ownerId, parentId?, name, kind | Unique owner + parent + name where suitable |
| codeWorkspaces | ownerId, videoId?, topicId?, files, language | Keep document size bounded |
| executionJobs | ownerId, status, language, limits, resultRef, expiresAt | Short retention for output/logs |
| aiJobs | jobType, targetId, version, status, attempts, idempotencyKey | Operational queue metadata |

## 11.1 Example note document

```
{
  "_id": "note_123",
  "ownerId": "user_123",
  "folderId": "folder_react",
  "source": {
    "type": "video",
    "videoId": "video_123",
    "timestampSeconds": 420
  },
  "type": "AI_GENERATED",
  "title": "React Hooks - useEffect",
  "topicIds": ["react_hooks", "use_effect"],
  "content": {
    "sections": [
      {
        "heading": "Purpose",
        "body": "Synchronise a component with an external system.",
        "sourceSegmentIds": ["seg_22"]
      }
    ]
  },
  "createdAt": "2026-11-06T06:00:00Z",
  "updatedAt": "2026-11-06T06:00:00Z"
}
```

## 11.2 Index strategy

| Collection | Candidate indexes | Purpose |
|---|---|---|
| videos | unique(provider, providerVideoId); status + publishedAt | Deduplication and catalog queries |
| topics | unique(slug); parentId + status | Taxonomy navigation |
| userInterests | unique(userId, topicId); userId + state | Load a user's followed topics |
| videoTopics | topicId + status/coverage; videoId | Topic candidate lookup |
| videoRatings | unique(userId, targetId, scope); targetId + createdAt | Prevent duplicate ratings and aggregate |
| playlistItems | unique(playlistId, position); playlistId + videoId | Ordered playlist reads |
| notes | ownerId + folderId + updatedAt; ownerId + topicIds | Library browsing |
| aiJobs | status + nextRunAt; unique(idempotencyKey) | Worker scheduling and deduplication |

Indexes should follow measured query patterns. Too many indexes increase write and storage costs. Use explain plans and production query statistics; do not index every field by default.

## 11.3 Data ownership and access

- Every user-owned record includes ownerId and is checked server-side on every read/write.
- Public playlist visibility does not make private notes or workspace files public.
- Use explicit DTOs to avoid accidentally returning internal fields.
- Use soft deletion only where product recovery requirements justify it.
- Define retention for transcript caches, code execution output, logs and AI prompts.

# 12. API design and endpoint catalogue

Use a versioned REST API initially. Validate request bodies, query parameters and IDs at the boundary. Return consistent errors, pagination cursors and request IDs. The endpoint list below is a starting contract; refine response schemas during implementation.

| Area | Method and route | Purpose |
|---|---|---|
| Interests | GET /api/v1/me/interests | List followed topics |
| Interests | PUT /api/v1/me/interests/:topicId | Follow/update interest |
| Interests | DELETE /api/v1/me/interests/:topicId | Unfollow topic |
| Taxonomy | GET /api/v1/topics?parentId= | Browse taxonomy |
| Discovery | GET /api/v1/feed?cursor= | Load focused feed |
| Discovery | GET /api/v1/topics/:topicId/videos | Topic-ranked videos |
| Discovery | GET /api/v1/search?q= | Search videos, topics, playlists |
| Videos | GET /api/v1/videos/:videoId | Video detail and availability |
| Ratings | PUT /api/v1/videos/:videoId/ratings | Create/update user's rating |
| Ratings | GET /api/v1/videos/:videoId/ratings/summary | Rating aggregate |
| Playlists | POST /api/v1/playlists | Create user playlist |
| Playlists | GET /api/v1/playlists/:id | Playlist details/items |
| Playlists | PATCH /api/v1/playlists/:id/items/reorder | Reorder items |
| Playlists | POST /api/v1/topics/:id/ai-playlists | Request AI playlist generation |
| Playlists | POST /api/v1/playlists/:id/ratings | Rate playlist |
| Workspace | GET /api/v1/workspaces/:id | Load code workspace |
| Workspace | PUT /api/v1/workspaces/:id/files | Save code files |
| Execution | POST /api/v1/code/execute | Create sandbox execution job |
| Execution | GET /api/v1/code/jobs/:jobId | Poll execution status/result |
| Notes | POST /api/v1/notes | Create personal or AI note |
| Notes | GET /api/v1/notes?folderId= | List notes |
| Notes | PATCH /api/v1/notes/:id | Edit note |
| Notes | DELETE /api/v1/notes/:id | Delete note |
| Folders | POST /api/v1/folders | Create folder |
| Folders | PATCH /api/v1/folders/:id | Rename/move folder |
| AI | POST /api/v1/ai/notes | Request note generation |
| AI | POST /api/v1/ai/ask | Ask grounded question |
| Jobs | GET /api/v1/jobs/:jobId | Check asynchronous AI job |

## 12.1 API conventions

- Use cursor pagination for feeds and large lists; avoid deep page-number offsets.
- Use idempotency keys for job creation and payment-like operations if introduced.
- Use PATCH for partial updates and PUT for full replacement semantics.
- Return 202 Accepted for background AI or execution jobs.
- Return 401 for unauthenticated requests, 403 for forbidden resources, 404 when resource is not visible, 409 for conflicts, 422 for invalid semantic input, and 429 for rate limits.
- Use consistent error objects: code, message, requestId, fieldErrors where safe.

### 12.2 Example API response

```
{
  "data": {
    "jobId": "job_abc",
    "status": "QUEUED",
    "statusUrl": "/api/v1/jobs/job_abc"
  },
  "meta": {
    "requestId": "req_xyz"
  }
}
```

# 13. AI endpoint and job design

| Endpoint / job | Input | Output / behaviour |
|---|---|---|
| POST /ai/classify-video | Video metadata + permitted transcript chunks | Topic IDs, aliases, confidence, evidence segment IDs |
| POST /ai/evaluate-topic | Video ID + topic ID + evidence | Coverage, clarity, examples, gaps, limitations |
| POST /ai/generate-playlist | Topic ID + candidate video IDs + constraints | Ordered items, coverage, gaps, rationale |
| POST /ai/generate-notes | Video ID + authorised transcript/material | Structured notes and timestamps |
| POST /ai/ask | Question + video/topic context | Grounded answer with segment references |
| POST /ai/summarize-segment | Selected segment IDs | Short summary of selected content |
| POST /ai/extract-code | Authorised transcript segment IDs | Code snippets with language and source references |
| POST /ai/create-quiz | Topic + permitted source material | Questions, answers and explanations |
| POST /ai/moderate-content | Content ID + text/metadata | Flags, severity and review recommendation |
| POST /ai/reprocess | Target ID + evaluation version | Queues re-evaluation with idempotency |

### 13.1 AI job lifecycle

### 13.2 Grounded learning assistant

The Ask AI feature should answer from the current video's authorised transcript, selected transcript segments, or approved topic materials. Return citations to timestamps/segment IDs. If the source does not answer the question, say so and offer a broader topic search only if the user requests it. Never pretend that an answer came from the video when it did not.

### 13.3 Human review and feedback loop

- Admin review queue for low-confidence topic mapping and serious accuracy flags.
- Users can report an incorrect AI note or topic label.
- Store prompt/model/schema versions for reproducibility.
- Maintain a small gold-standard dataset of manually reviewed videos and expected topic coverage.
- Run evaluation tests before changing prompts or model versions.
- Allow reprocessing without overwriting user-edited notes.

# 14. Website and application optimisation

## 14.1 Performance budget

Set budgets before adding features. Example initial targets: mobile LCP under 2.5 seconds, INP under 200 ms, CLS under 0.1 at the 75th percentile where measurable. Treat these as targets, not guarantees. Measure real-user performance by route and device.

## 14.2 Frontend optimisation

- Use server components for data-heavy read views where appropriate; keep client components limited to interactive workspace pieces.
- Lazy-load the code editor, transcript tools and AI panels until opened.
- Do not bundle Monaco into the initial feed page.
- Use responsive image sizing, modern formats and explicit dimensions to prevent layout shifts.
- Use skeletons that match final layout; avoid giant loading spinners.
- Virtualise long lists of videos, notes and playlist items.
- Debounce search input and cancel stale requests.
- Use route-level loading and error boundaries.
- Avoid storing duplicate server data in multiple client state stores.
- Keep the player stable when switching tabs so playback does not restart unnecessarily.

## 14.3 Backend optimisation

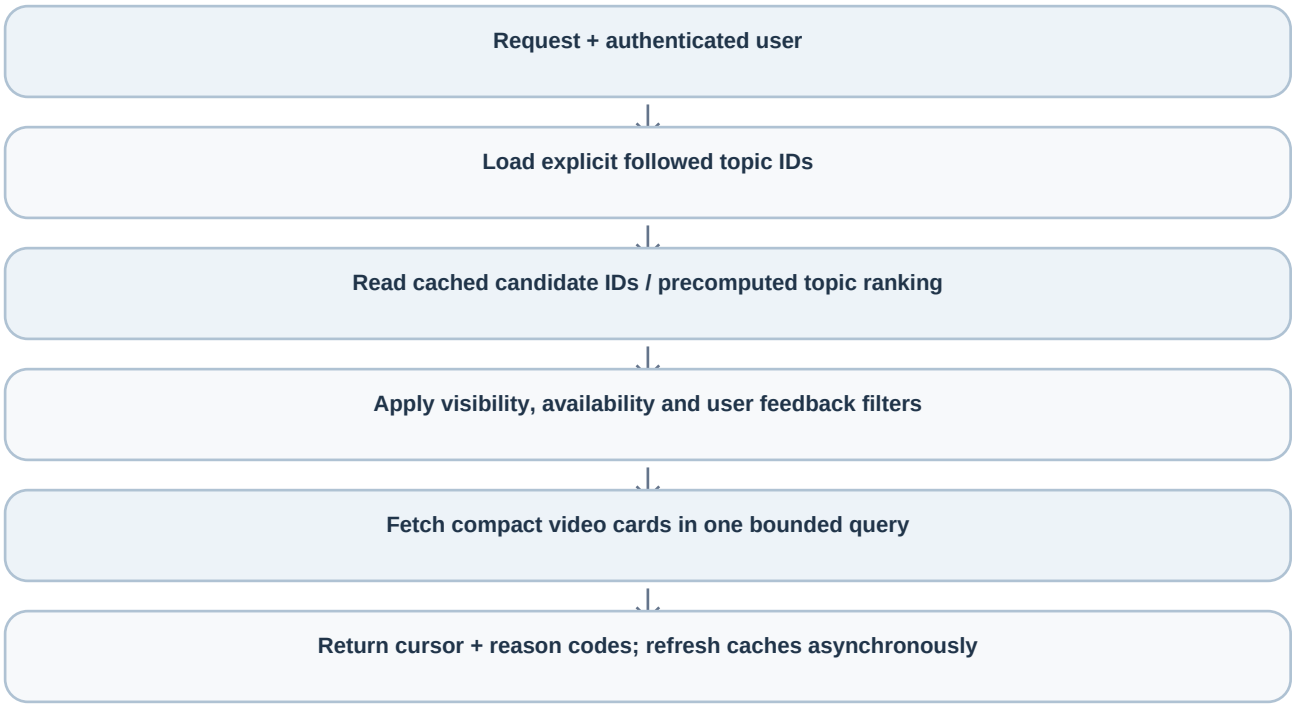
- Use projections to fetch only fields required for feed cards.
- Use cursor pagination and stable sort keys.
- Batch related reads to avoid N+1 query patterns.
- Cache topic aggregates and ranking results with explicit invalidation.
- Precompute topic ranking snapshots rather than aggregating every rating on every request.
- Move transcript processing, embeddings and AI generation to workers.
- Use connection pooling and set sensible timeouts.
- Compress responses where beneficial; avoid compressing already-compressed media.
- Add request budgets and per-route rate limits.

## 14.4 MongoDB optimisation

Design indexes around real query shapes. Use explain plans, query statistics and slow-query logs. Avoid unbounded arrays inside user or video documents. Keep large transcripts, logs and code outputs in separate collections or object storage. Use TTL indexes only for data with a clearly defined expiry policy.

## 14.5 Feed performance design

### FAST FEED REQUEST PATH




The feed request should not call the LLM. The LLM enriches the catalogue offline; ranking code serves precomputed signals quickly. Invalidate or refresh a topic ranking when ratings, evaluations or availability change.

## 14.6 AI cost optimisation

| Technique | How to use it |
|---|---|
| Deduplication | Same video/transcript/version should not be evaluated repeatedly |
| Caching | Reuse classification, embeddings, notes templates where safe |
| Batch API | Use for non-urgent bulk classification/evaluation/embedding work |
| Model routing | Use lower-cost models for simple extraction; stronger models for ambiguous assessments |
| Chunking | Send only relevant transcript chunks for a topic-specific task |
| User quotas | Limit repeated expensive generations and provide transparent usage |
| Async processing | Queue generation and return a job ID |
| Evaluation gates | Do not regenerate when the source hash and prompt version are unchanged |

## 14.7 Observability

- Frontend: Web Vitals, JS errors, route load time, editor load time.
- Backend: request latency percentiles, error rate, DB query latency, queue age.
- AI: token usage, cost per job, latency, refusal/error rate, schema failures and user correction rate.

- Sandbox: queue time, execution duration, timeout rate, memory usage and abuse signals.
- Product: feed relevance, save rate, playlist completion and workspace adoption.

# 15. Security, privacy and trust

## 15.1 Authentication and authorization

- Use secure session handling and modern password hashing if password auth is supported.
- Enforce object-level authorization for every user-owned note, folder, playlist and code workspace.
- Use role-based permissions for admin and moderator operations.
- Do not trust ownerId, role or visibility fields supplied by the browser.
- Protect sensitive actions with re-authentication where appropriate.

## 15.2 API security

Use the OWASP API Security Top 10 as a baseline. Important risks for this product include broken object-level authorisation, broken authentication, unrestricted resource consumption, unsafe third-party API consumption and security misconfiguration.

## 15.3 Prompt injection and untrusted content

- Treat transcripts, comments, descriptions and user notes as untrusted data, not instructions.
- Separate system instructions from source content and explicitly tell the model to analyse source text rather than follow it.
- Do not let transcript content trigger tool calls or database actions.
- Use allowlisted tools and server-side authorization for any AI tool invocation.
- Validate model outputs and never trust generated URLs, IDs or permission decisions.

## 15.4 Privacy and data minimisation

- Tell users what data is sent to AI providers and why.
- Avoid sending account details or private notes unless required for the requested feature.
- Define retention and deletion rules for transcripts, prompts, code and execution output.
- Do not use private notes to train recommendation models without clear user permission and policy.
- Offer export and account deletion workflows appropriate to applicable law.

## 15.5 Content rights and platform policy

Before launch, review YouTube API Services Terms and Developer Policies, copyright/licensing rules, privacy obligations and applicable consumer protection rules. Embedding a video does not automatically grant rights to download, transcribe, rehost or create derivative copies of its content. Get qualified legal advice for the actual business model and jurisdictions.

# 16. Admin and content operations

| Admin capability | Purpose |
|---|---|
| Taxonomy manager | Create, merge, rename, alias and retire topics |
| Video review queue | Review low-confidence classifications and duplicate matches |
| AI evaluation review | Inspect evidence, scores, model version and reported errors |
| Playlist review | Inspect public playlist abuse and quality reports |
| User reports | Handle spam, copyright, misleading content and safety reports |
| Job monitor | Retry failed ingestion, AI and embedding jobs |
| System dashboard | Queue age, API errors, AI cost, ranking health and usage |

Admin tools should be permissioned and audited. Record who changed a topic mapping, removed a public playlist or overrode an AI evaluation, with timestamp and reason.

# 17. Testing and quality strategy

| Test layer | Examples |
|---|---|
| Unit | Ranking formula, Bayesian rating, topic mapping rules, note validation |
| Integration | API + MongoDB ownership checks, playlist reorder, job retries |
| Contract | LLM structured output schemas, YouTube API adapter, sandbox adapter |
| End-to-end | Follow topic → feed → watch → note → save → revisit |
| Security | IDOR, auth bypass, rate limits, prompt injection, sandbox escape review |
| Performance | Feed at target concurrency, long playlist, large note library |
| AI evaluation | Topic classification precision, evidence correctness, note factuality, playlist coverage |
| Accessibility | Keyboard navigation, screen reader labels, contrast, mobile layout |

## 17.1 Ranking evaluation

Build a small hand-labelled dataset: videos, target topics, expected topic coverage and expert/learner preference judgements. Compare ranking versions offline using metrics such as NDCG@K, precision@K and topic coverage. Then run a limited experiment with clear rollback. Do not optimise only for watch time; it may reward addictive or irrelevant content.

## 17.2 AI quality gates

- Schema validity must be 100% after server validation.
- Evidence segment IDs must exist and belong to the supplied video.
- Timestamp references must fall within video duration.
- Unknown/insufficient evidence must remain unknown, not be guessed.
- Sample generated notes for factual correctness before expanding generation volume.
- Track user edits and reports as quality signals.

# 18. Phased development roadmap

| Phase | Scope | Exit criteria |
|---|---|---|
| 0. Product foundation | User stories, taxonomy, wireframes, provider/policy review, data model | Scope agreed; core journeys clickable in prototype |
| 1. Core platform | Auth, profile, topic taxonomy, interest selection, video catalog, focused feed | User can follow topics and see relevant videos |
| 2. Topic discovery | Topic pages, video-topic mapping, search, feedback controls | Videos appear under correct topics; users can correct relevance |
| 3. Ratings and ranking | Video ratings, confidence-adjusted aggregates, topic ranking, reason codes | Ranking is explainable and testable |
| 4. Playlists | AI-ranked playlist, custom playlist, reorder, save/fork, playlist rating | User can create and customise a learning path |
| 5. Workspace basics | Embedded player, personal notes, timestamps, folders, library | User can take and retrieve notes while watching |
| 6. AI content features | Authorised transcript pipeline, AI evaluation, AI notes, job status | AI output is structured, evidenced and editable |
| 7. Developer practice | Editor, saved code, isolated execution, quotas | User can safely run supported languages |
| 8. Scale and polish | Caching, search improvements, analytics, moderation, accessibility | Measured performance and operational readiness |

## 18.1 Suggested solo-developer order

Build the smallest end-to-end vertical slice first: user follows React Hooks → sees topic page → opens embedded video → writes a timestamped note → saves it. Then add ratings and ranking. Then playlists. Then AI notes. This avoids spending weeks building AI pipelines before validating the central user experience.

## 18.2 Example 12-week plan

| Weeks | Deliverable |
|---|---|
| 1–2 | Product specification, UX wireframes, taxonomy, auth, project foundation |
| 3–4 | Video catalog, topic pages, interest selection, focused feed |
| 5–6 | Ratings, aggregates, topic ranking, search and feedback |
| 7–8 | User playlists, AI playlist prototype, reorder and share |
| 9–10 | Video workspace, timestamped notes, folders and library |
| 11 | Authorised transcript flow and AI note generation |
| 12 | Testing, performance, security review, deploy and pilot |

This is an aggressive planning estimate for a focused MVP, not a delivery guarantee. A solo developer may need more time, especially for content rights, sandbox hardening, AI evaluation and operational tooling.

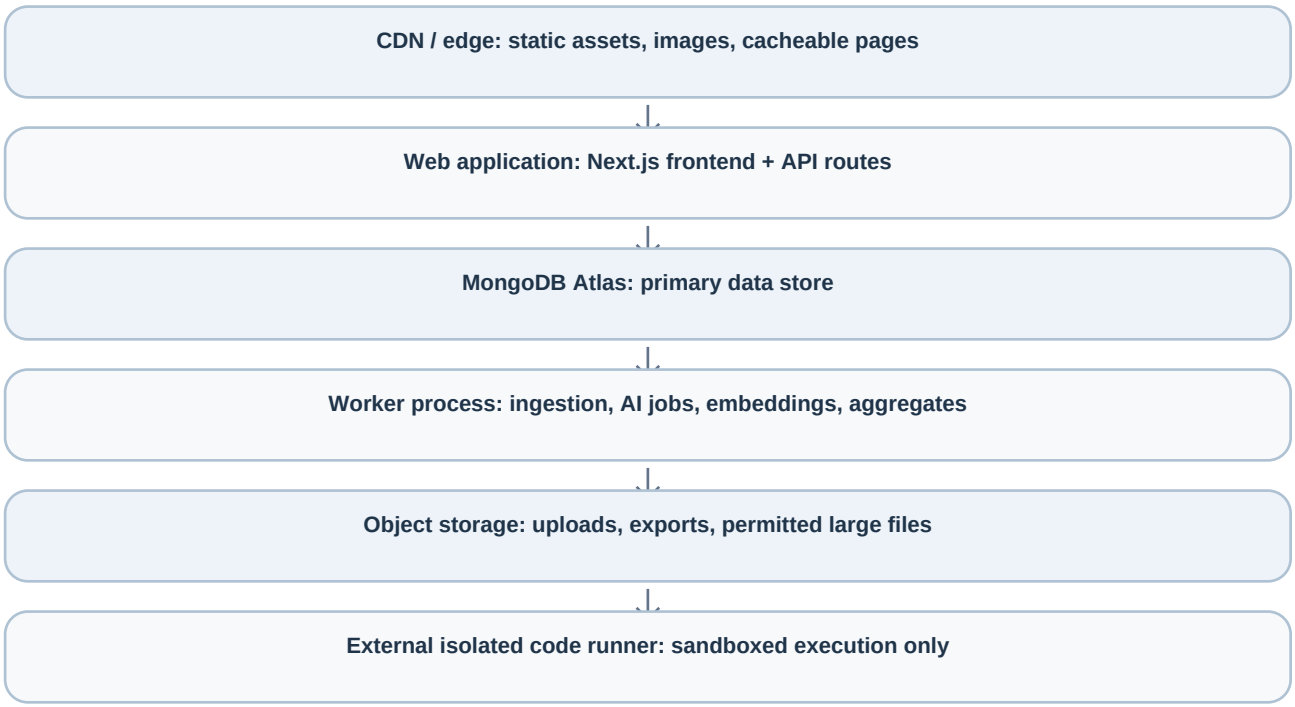
# 19. Deployment, environments and operations

## 19.1 Environments

| Environment | Use |
|---|---|
| Local | Development with local database or isolated dev Atlas cluster |
| Preview | Pull-request validation and UX review |
| Staging | Production-like integration, sandbox and AI testing |
| Production | Real users, monitoring, backups, rate limits and incident process |

## 19.2 Deployment layout

### INITIAL DEPLOYMENT




## 19.3 Operational basics

- Use separate API keys and secrets per environment.
- Store secrets in deployment secret management, never in source control.
- Enable automated database backups and test restoration.
- Add health checks for web app, worker and queue.
- Use structured logs with request IDs; redact tokens and private content.
- Create alerts for elevated error rates, queue backlog, database saturation and unusual AI spend.
- Have rollback procedures for deployments, prompt versions and ranking weights.

# 20. Cost model and budget planning

Costs depend on usage, model choice, video source, storage, sandbox execution, region and traffic. Treat the following as cost categories to measure, not fixed vendor quotes. Obtain current pricing before committing.

| Cost area | Main cost drivers | How to control |
|---|---|---|
| Web hosting | Requests, server execution, bandwidth, build minutes | Cache static/read pages, avoid oversized server functions |
| Database | Compute tier, storage, backups, network and indexes | Start small, measure query load, avoid unnecessary indexes |
| AI APIs | Input/output tokens, embeddings, retries, model selection | Batch, cache, chunk, route models, quota expensive features |
| Video APIs | Quota and permitted API operations | Cache policy-compliant metadata, avoid repeated searches |
| Storage/CDN | User uploads, thumbnails, exports, bandwidth | Compress, resize, lifecycle policies |
| Code execution | Concurrent jobs, CPU, memory, isolation infrastructure | Per-user limits, timeouts, supported languages, queue |
| Monitoring | Events, logs, traces and retention | Sampling, redact, set retention budgets |

## 20.1 Track unit economics

- Cost per video ingested and classified.
- Cost per topic evaluation.
- Cost per AI note generated.
- Cost per active learner per month.
- Sandbox cost per successful execution.
- Database and bandwidth cost per 1,000 feed views.

**Do not launch unlimited AI note generation or unlimited code execution on a free tier without quotas. These are variable-cost features and can be abused.**

## 20.2 Tax / compliance clarification

This blueprint assumes 'tax' in the request meant 'tech stack'. If the intended request was taxation, do not rely on this product document for tax advice. An India-based business may need a CA to assess GST registration and place-of-supply treatment, income tax, invoicing, foreign SaaS/API payments, withholding and export-of-services issues based on the actual legal entity, revenue model and customer locations.

# 21. Monetisation options and product boundaries

| Model | Potential fit | Trade-off |
|---|---|---|
| Free core + paid AI credits | Basic feed and notes free; AI notes/assistant limited | Simple to explain; requires usage metering |
| Subscription | Higher AI quota, advanced workspace and exports | Recurring revenue; must deliver recurring value |
| Creator/partner plans | Creator analytics, playlist publishing tools | Requires creator acquisition and moderation |
| Institution/team plans | Shared learning paths and admin controls | Adds permissions, billing and support complexity |

Avoid monetisation that compromises ranking integrity. If sponsored content is introduced, label it distinctly and keep it separate from organic learner ratings and AI quality scores.

# 22. Pilot launch and product analytics

## 22.1 Pilot cohort

Start with one or two content areas where you can validate topic mapping and learning workflows. For example, DSA and React/Web Development are suitable initial areas for a developer-focused pilot. Do not attempt to populate every subject category before proving the core loop.

## 22.2 Events to instrument

| Event | Useful properties |
|---|---|
| topic_followed | topicId, source, timestamp |
| feed_impression | videoId, topicId, rank, rankingVersion |
| video_opened | videoId, topicId, sourceSurface |
| video_progress | videoId, secondsWatched, playbackState |
| video_rated | videoId, topicId?, rating, feedback |
| playlist_created | playlistId, type, topicId |
| playlist_reordered | playlistId, itemCount |
| note_created | noteType, folderId, sourceType |
| ai_note_generated | modelVersion, tokenUsage, latency, cost |
| code_run | language, duration, status, resourceClass |
| not_interested | videoId, topicId, reason |

Avoid collecting more behavioural data than needed. Document event purpose, retention and user controls. Use aggregate analytics for product decisions wherever possible.

# 23. Key risks and mitigation

| Risk | Impact | Mitigation |
|---|---|---|
| Transcript unavailable or unauthorised | AI features cannot run on many videos | Creator-provided/licensed materials; graceful fallback |
| AI evaluation hallucination | Incorrect ranking and learner trust loss | Evidence references, confidence, human review, eval dataset |
| Popular videos dominate | New useful explanations never surface | Confidence-adjusted ratings, exploration slot, freshness |
| Rating manipulation | Unfair rankings | Rate limits, anomaly detection, audit trails |
| Taxonomy becomes too complex | Poor discovery and admin burden | Start small; aliases, stable IDs, usage-led taxonomy |
| Code sandbox abuse | Security incident or unexpected cost | Dedicated isolated runner, quotas, network isolation |
| AI cost spikes | Unsustainable unit economics | Quotas, batch jobs, caching, model routing |
| Mobile workspace friction | Low adoption outside desktop | Responsive tool tabs; lightweight editor on mobile |
| Platform policy changes | Ingestion or playback disruption | Provider abstraction, policy monitoring, licensed content options |
| Overbuilding before validation | Long time to launch | Vertical-slice MVP and staged rollout |

# 24. Recommended product decisions

| Decision | Recommendation | Reason |
|---|---|---|
| Feed policy | Strict followed-topic feed by default; separate Explore | Preserves the promise of reduced distraction |
| Interest adaptation | Explicit user controls; suggestions require approval | Avoids accidental watch behaviour changing interests |
| Ranking | Combine user ratings, topic coverage, AI signals and trust | Balances community evidence and content understanding |
| Rating | Separate video, topic-specific and playlist ratings | A video may be strong for one concept but not another |
| Playlist | AI-generated + user-created + customisable AI copy | Combines automation with user ownership |
| Notes | Keep personal and AI notes separate; allow merge/copy | Prevents AI output from overwriting personal work |
| Code | Editor UI plus separate sandbox execution service | Security boundary and independent scaling |
| Architecture | Modular monolith + background worker first | Appropriate for a solo developer |
| AI | Structured outputs, evidence references, versioning | Reliable integration and debuggability |
| Content | Embed first; use only permitted metadata/transcripts | Reduces rights and infrastructure risk |

# 25. Development checklist

## Product foundation

- [ ] Define first pilot subject areas and topic hierarchy
- [ ] Write user stories for feed, topic, playlist, workspace and library
- [ ] Create desktop and mobile wireframes
- [ ] Decide public/private playlist rules
- [ ] Document content-source and transcript permissions

## Core application

- [ ] Set up Next.js, TypeScript, linting, tests and CI
- [ ] Implement authentication and user ownership checks
- [ ] Create topic and user-interest APIs
- [ ] Create video catalog and ingestion adapter
- [ ] Build focused feed and topic detail pages

## Ranking and playlists

- [ ] Implement rating model and duplicate-rating protection
- [ ] Implement Bayesian/confidence-adjusted aggregate
- [ ] Create topic ranking service and reason codes
- [ ] Build AI and user playlist types
- [ ] Implement reorder, fork/copy, visibility and playlist ratings

## Workspace and AI

- [ ] Integrate embedded player and timestamp events
- [ ] Build personal notes and autosave
- [ ] Build folders and library
- [ ] Implement authorised transcript processing
- [ ] Build versioned AI evaluation and note generation jobs
- [ ] Add evidence links and user correction/reporting

## Developer tools

- [ ] Integrate editor with lazy loading
- [ ] Build code workspace save/load
- [ ] Integrate isolated code execution service
- [ ] Set CPU, memory, time, output and concurrency limits
- [ ] Add usage quotas and abuse monitoring

## Launch readiness

- [ ] Add observability and cost dashboards
- [ ] Test access control and prompt injection handling
- [ ] Test mobile layout and accessibility

- [ ] Run ranking and AI evaluation datasets
- [ ] Test database backup and restore
- [ ] Pilot with a small user group and iterate

# 26. Glossary

| Term | Meaning in this product |
|---|---|
| Topic coverage | How much of a defined topic or concept a video actually explains |
| Evidence segment | A transcript/material segment supporting an AI assessment |
| Bayesian rating | An average adjusted to account for small sample sizes |
| Embedding | A vector representation used for semantic similarity and retrieval |
| RAG | Retrieval-augmented generation: retrieve relevant source material before asking AI |
| Idempotency | Repeating the same request/job does not create duplicate effects |
| Cursor pagination | Pagination using a stable continuation key rather than large offsets |
| Modular monolith | One deployable application separated into well-defined internal modules |
| INP / LCP / CLS | Core Web Vitals measures for interaction, loading and layout stability |
| IDOR / BOLA | Access-control flaws where a user can access another user's object by changing an identifier |

# 27. Official technical references

The following official or primary-source references were checked while preparing this blueprint. APIs, policies and pricing can change; re-check documentation immediately before implementation and launch.

**OpenAI Structured Outputs** — https://developers.openai.com/api/docs/guides/structured-outputs

Use schema-constrained outputs for predictable AI integration; validate semantic correctness separately.

**OpenAI Batch API** — https://developers.openai.com/api/docs/guides/batch

Useful for asynchronous bulk evaluation, classification and embedding jobs.

**OpenAI API overview** — https://developers.openai.com/api/reference/overview

Current API surfaces, authentication, requests, errors and rate limits.

**YouTube Data API getting started** — https://developers.google.com/youtube/data/getting-started

API operations, quota and access model.

**YouTube API developer policies** — https://developers.google.com/youtube/terms/developer-policies

Required policy review for API clients.

**YouTube IFrame Player API** — https://developers.google.com/youtube/iframe_api_reference

Embedded player integration and playback events.

**YouTube captions API** — https://developers.google.com/youtube/v3/docs/captions

Caption resource and authorisation requirements.

**MongoDB index design** — https://www.mongodb.com/docs/manual/data-modeling/schema-design-process/create-indexes/

Index planning and query coverage.

**MongoDB Atlas Search** — https://www.mongodb.com/docs/atlas/atlas-search/

Search architecture and indexing.

**Monaco Editor** — https://microsoft.github.io/monaco-editor/

Browser editor capabilities and mobile-browser limitation.

**Judge0 CE API docs** — https://ce.judge0.com/docs

Sandboxed compilation/execution API and resource limits.

**OWASP API Security Top 10** — https://owasp.org/API-Security/editions/2023/en/0x11-t10/

API security risks and mitigation guidance.

**Next.js documentation** — https://nextjs.org/docs

Framework features, rendering, caching and deployment guidance.

**web.dev Core Web Vitals** — https://web.dev/articles/vitals

Performance metrics and user experience targets.

# 28. Final recommendation

Build the product around a tight learning loop: choose a topic → discover a strong explanation → watch and practise → capture notes → save a learning path → give feedback. This loop should work well before you scale the catalogue or add every AI feature.

The strongest first release is not the one with the most AI endpoints. It is the one that reliably shows relevant educational videos, makes topic rankings understandable, lets users organise a useful playlist, and keeps notes connected to the moment they learned something.

**Recommended first milestone: a working React Hooks or DSA topic experience with a focused feed, topic-specific ratings, a custom playlist, embedded playback, timestamped notes and a personal folder. Add AI classification and generated notes once the content-permission and evaluation pipeline is dependable.**

This blueprint is a product and engineering plan. It is not a substitute for a security audit, legal review, provider policy review or tax advice.